STATE UNIVERSITY OF NEW YORK AT BINGHAMTON

Stock Insights: AI-Powered Forecasting and Sentiment Analysis Platform

Under the guidance of ***Professor Adnan Rakin Siraj***

Student Name: Naga Ramya Gurrala
B-Number: B01040125
Advisor Name: Professor Adnan Rakin Siraj
Date of Completion: May 07, 2025
GitHub Repository: `https://github.com/NagaRamya1531-tech/stock_project`

Abstract

Stock Insights is an AI-driven, web-based platform designed to forecast stock prices and visualize market sentiment using real-time financial and social media data. This project presents the design and development of a unified dashboard that combines predictive analytics with sentiment evaluation to provide users with actionable investment insights. The application leverages time-series forecasting models such as ARIMA, LSTM, and Linear Regression to generate accurate future price predictions. Complementing this, it scrapes public sentiment from platforms like Twitter, Reddit, and 4chan using natural language processing (VADER) to derive behavioral insights into market perception. These two analytical components—technical forecasting and sentiment analysis—work together to generate intuitive signals such as "Buy," "Sell," or "Hold."

The platform is developed using Python and Flask, with core libraries such as Alpha Vantage API (for real-time stock data), Keras (for deep learning models), BeautifulSoup (for web scraping), Plotly and Seaborn (for data visualization). Users can register and log in securely, enter stock ticker symbols, and receive detailed forecast outputs in the form of interactive charts, accompanied by sentiment distributions in visual formats like pie charts and bar graphs. The system integrates backend model processing, frontend visualization, and data communication into one seamless interface.

By merging financial modeling, machine learning, and real-time sentiment analysis, Stock Insights offers an accessible and intelligent tool for investors, students, and researchers. Its modular architecture allows for easy expansion into future features such as portfolio monitoring, volatility tracking, and automated alerts, laying the groundwork for further innovation in the fintech space.

1 Introduction

Financial markets are inherently dynamic, influenced by a wide range of factors including corporate performance, geopolitical developments, economic indicators, and investor sentiment. Traditional financial platforms often cater to professional analysts and institutional investors, leaving a knowledge and accessibility gap for students, researchers, and everyday retail investors.

This project addresses that gap by building Stock Insights, a user-friendly, intelligent, and interactive stock market dashboard. The platform integrates multiple modules to deliver a comprehensive suite of tools:

- Stock price exploration with performance metrics and historical visuals
- Forecasting tools using AI models like LSTM and Arima and Linear Regression
- Sentiment analysis based on real-time Twitter, Reddit, and 4chan feeds
- Comparative analysis for evaluating multiple equities
- Real-time visualization with candlestick, line, and OHLC charts
- Market trends highlighting top gainers, losers, and global indices

2 Significance of the Project

Stock Insights addresses a critical gap in the accessibility of intelligent financial tools for non-professional users. While institutional platforms offer powerful forecasting and sentiment analysis features, they are often expensive, complex, or closed off to the public. This project bridges that divide by providing a free, modular platform that empowers students, early investors, and researchers to analyze the market from both technical and behavioral perspectives.

What makes this project significant is its integration of two traditionally separate approaches—machine learning–based forecasting and sentiment-driven insights—into one coherent, user-friendly dashboard. The platform enables users not only to predict future stock movements using models like LSTM, Prophet, ARIMA, and Linear Regression, but also to understand how public mood on platforms like Twitter, Reddit, and 4chan may influence those movements.

Beyond functionality, the project has academic and technical value. It demonstrates how diverse fields such as financial modeling, natural language processing, and full-stack web development can be unified in a real-world application. Its architecture is scalable and open-source, making it a viable foundation for future enhancements like portfolio tracking, real-time trading integration, or the use of advanced transformer-based sentiment models.

In short, Stock Insights is a step toward democratizing data-driven investment analysis. It supports learning, research, and informed decision-making—delivering real utility while showcasing the applied potential of artificial intelligence in finance.

3 Project Details

3.1 Design Overview

Stock Insights was developed as a modular, full-stack web application with a focus on usability, scalability, and data-driven functionality. The platform is divided into several components—each responsible for a specific task such as data collection, processing, modeling, visualization, or user interaction. The design separates backend logic from the frontend UI, allowing for clean integration of machine learning and sentiment analysis modules.

Users interact with the system through a browser-based dashboard built using HTML, CSS (Bootstrap), and Plotly for chart rendering. Backend services, built using Python and Flask, handle user input, fetch live stock data, run forecasting models, and return results in real time. Data pipelines connect to the Alpha Vantage API for financial data and to web scraping/NLP modules for social sentiment.

3.2 Implementation

Technologies Used

- **Languages:** Python (backend), HTML/CSS (frontend), JavaScript (interactive elements)
- **Frameworks:** Flask (web app), Bootstrap (UI), Jinja2 (template rendering)
- **APIs & Data Sources:** Alpha Vantage (stock data), Twitter API, Reddit (via Pushshift), 4chan (scraped)

3.3 Libraries

- **Machine Learning:** Keras (LSTM), Facebook Prophet, scikit-learn, statsmodels (ARIMA)
- **NLP:** VADER (for sentiment scoring), BeautifulSoup (for scraping text data)
- **Visualization:** Plotly, Seaborn, Matplotlib
- **BeautifulSoup:** For scraping text from Reddit and 4chan.
- **Plotly, Seaborn, Matplotlib:** Graph generation and charting.
- **Folium:** Geographical visualization of stock trend locations.

3.4 Database and Deployment

- **SQLite:** Local lightweight database to store user credentials.
- **Deployment Scope:** Currently local; scope for deployment via Heroku or Render.

4 System Architecture

The application architecture is modular and separated into layers for clarity and scalability. Below is a flow diagram of the internal process.

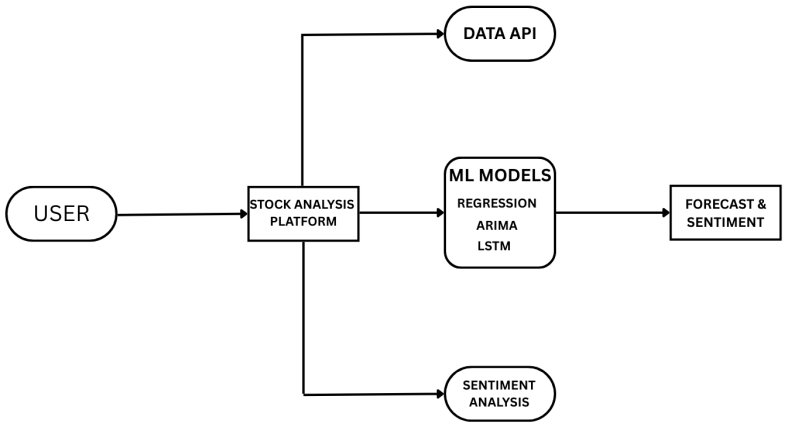

Figure 1: Architecture Diagram – Input to Output Flow

Explanation: The user enters a stock symbol and selects a date range. The system then fetches stock data, applies machine learning forecasting models, scrapes sentiment data, runs NLP classification, and returns both graphical price predictions and sentiment visuals via a Flask-rendered dashboard.

5 Application Modules Overview

The platform is divided into three functional modules to encapsulate different responsibilities.

5.1 Core Modules

- **Forecasting Module:** Implements Prophet, ARIMA, LSTM, and Linear Regression. Models are trained on historical closing prices to generate predictions for the next 7–30 days.
- **Sentiment Analysis Module:** Scrapes recent posts related to a selected stock from Twitter, Reddit, and 4chan. Text is cleaned and passed through VADER to determine sentiment polarity, which is then visualized using pie charts and bar graphs.
- **Visualization Module:** Allows users to choose between chart types (candlestick, OHLC, line, bar) and dynamically displays intraday or historical trends.
- **Stock Comparison Module:** Accepts two ticker symbols and compares them based on volatility, returns, and performance metrics using visual and tabular formats.

- **Authentication Module:** Users can register, log in, and explore personalized features securely using hashed credentials and session handling.

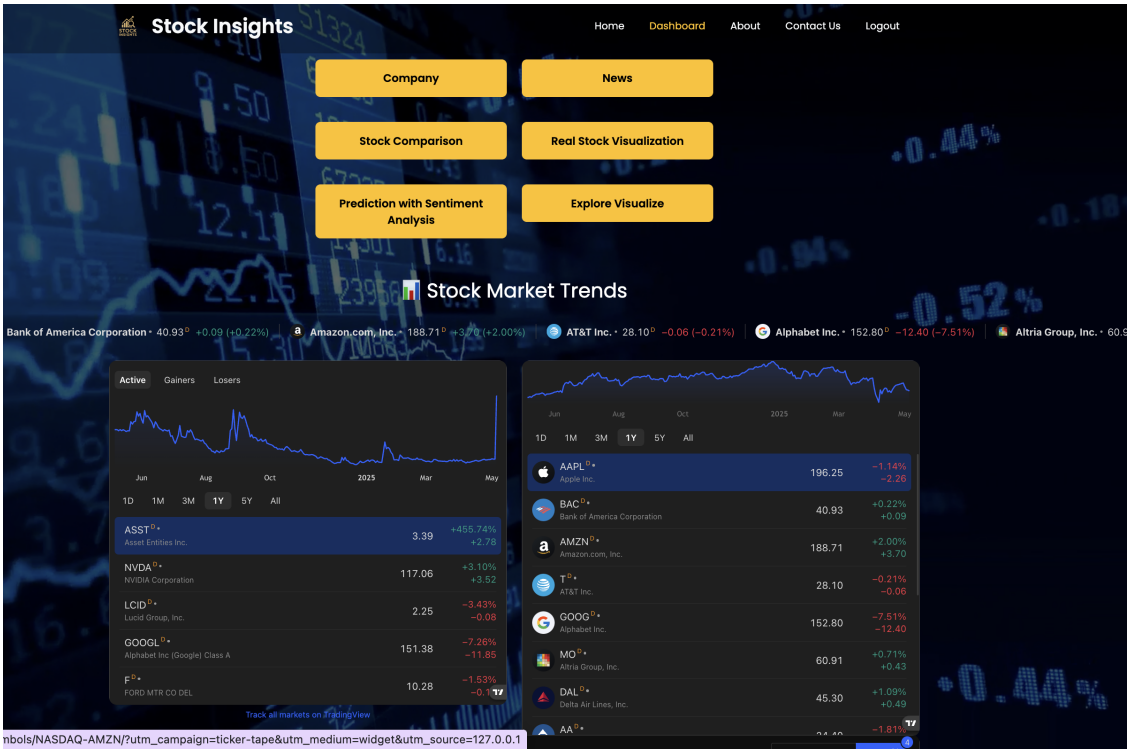

Figure 2: Dashboard

6 Forecasting Model Implementation

To provide robust predictions, this application integrates four time-series forecasting models. Each model is trained on historical stock prices and generates a future prediction curve which is then visualized for comparison.

6.1 1. LSTM

LSTM is a recurrent neural network model used for sequential data. It captures both short- and long-term dependencies.

Sample Code:

```
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(LSTM(50))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
```

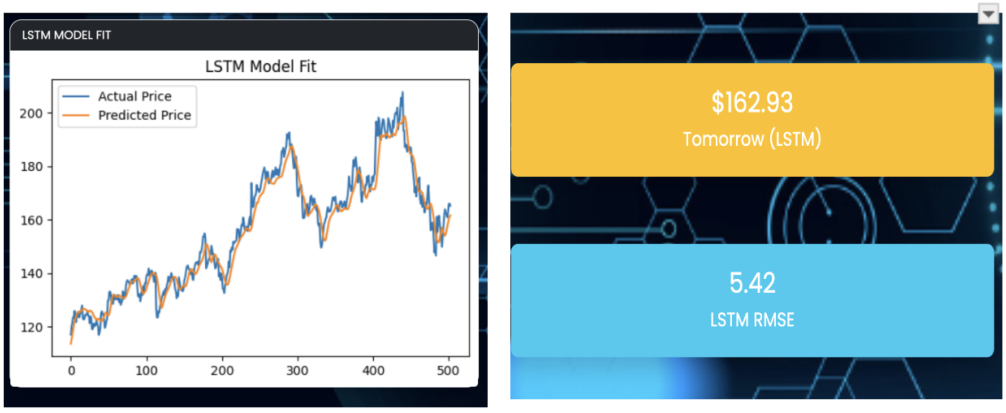

Figure 3: LSTM-Based Forecasting Output

6.2 2. ARIMA

ARIMA (AutoRegressive Integrated Moving Average) is a classic statistical model that excels at modeling autocorrelated data.

Sample Code:

```
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(df['Close'], order=(5,1,0))
model_fit = model.fit()
forecast = model_fit.forecast(steps=30)
```

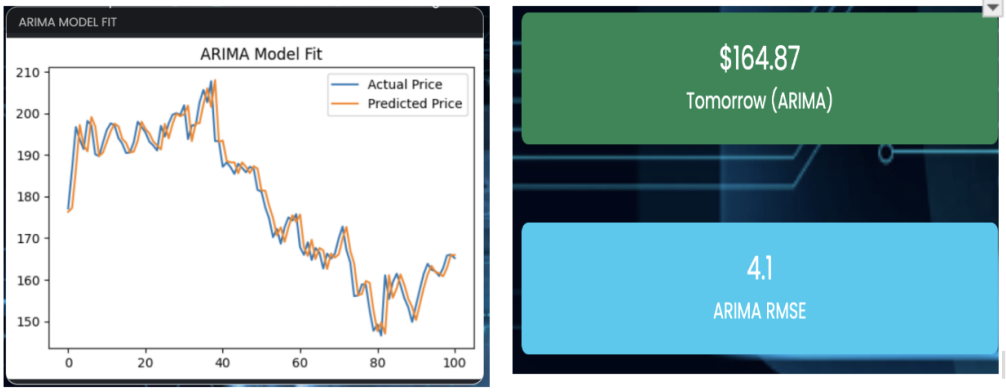

Figure 4: ARIMA Model Forecast Output

4. Linear Regression

A basic predictive model used as a baseline for comparison.

Sample Code:

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

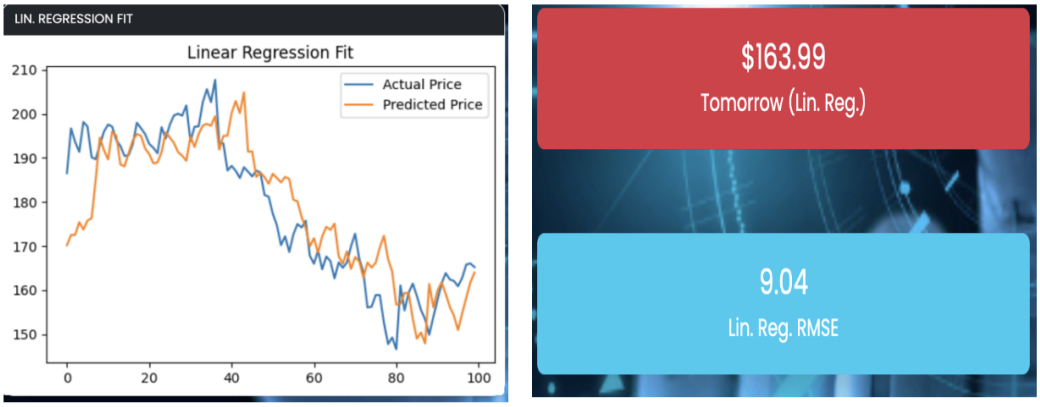

Figure 5: Forecast Output Using Linear Regression

7 Sentiment Analysis Module

To provide a behavioral layer to the technical forecast, this module collects and processes textual sentiment from various platforms:

Sources:

- Twitter via Tweepy
- Reddit using PRAW and web scraping
- 4chan via BeautifulSoup

Pipeline:

1. Scrape posts and comments using relevant libraries
2. Clean text (remove URLs, emojis, stopwords)
3. Tokenize and classify sentiment as Positive, Negative, or Neutral
4. Count and visualize the sentiment distribution

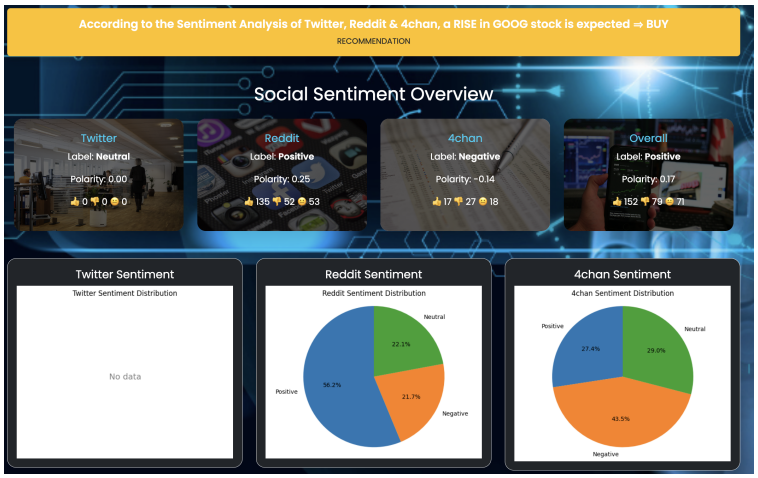

Figure 6: Sentiment Distribution Pie Chart

Example Output: "Sentiment Score: 67% Positive – Recommendation: HOLD"

8 Results and Evaluation

All forecasting models were tested using RMSE (Root Mean Square Error) as the metric for accuracy comparison. Based on our evaluation, the LSTM model delivered the most accurate results due to its ability to capture complex patterns. Linear Regression and ARIMA were also consistent for short- and medium-term trends.

Model	**RMSE**	**Remarks**
ARIMA	5.42	Suitable for stationary time series
LSTM	4.1	Best for non-linear and long-sequence data
Linear Regression	9.04	Baseline; fast but least accurate

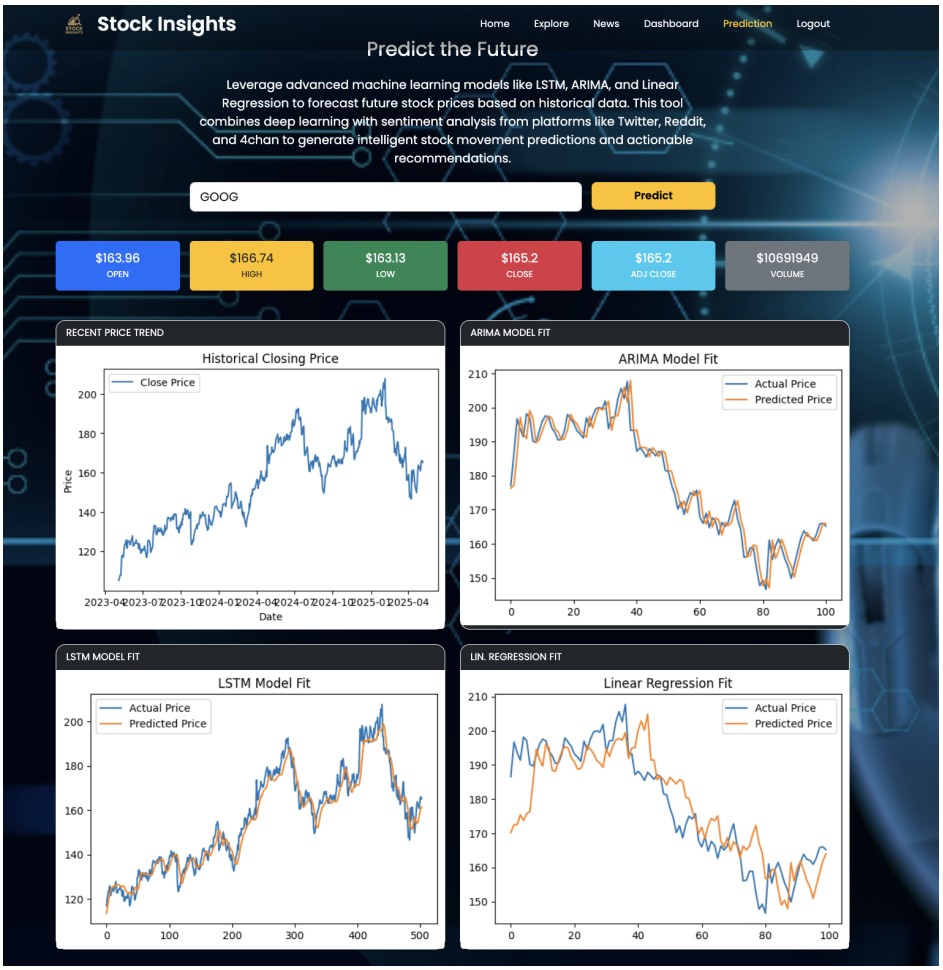

Figure 7: Comparison: Forecast vs Actual Price Across Models

9 Limitations

Despite the promising results, the project has several limitations:

- **Sentiment Noise:** Posts from social platforms often include sarcasm or unrelated content which basic NLP models struggle with.
- **Training Time:** LSTM models require GPU-based training for optimal performance.
- **API Limitations:** Rate limits from Twitter and Reddit APIs constrain real-time scalability.
- **Security:** The app does not yet support advanced authentication or CSRF protection.
- **Scalability:** Currently tested on a local host; not optimized for large user loads or cloud environments.

10 Conclusion

This project successfully integrates machine learning forecasting and sentiment analysis into a unified financial tool. The web application is capable of:

- Predicting future stock prices using multiple ML models
- Extracting and classifying market sentiment from public platforms
- Visualizing outputs in a digestible and interactive way

Through this, users gain both quantitative and qualitative insights, improving decision-making in trading scenarios. It also served as a strong educational foundation in web development, ML pipelines, and real-time data processing.

11 Future Scope

- Deploy the app on Heroku or AWS EC2 for public access
- Integrate with real-time trading APIs for live transactions
- Use BERT or transformer-based sentiment models
- Enable user-based portfolio customization
- Add cryptocurrency and index forecasting support

References

1. Keras LSTM: `https://keras.io/`
2. yfinance API: `https://pypi.org/project/yfinance/`
3. Alpha Vantage API: `https://www.alphavantage.co/documentation/`
4. ARIMA (statsmodels): `https://www.statsmodels.org/`
5. Flask Documentation: `https://flask.palletsprojects.com/`
6. Plotly Visualization: `https://plotly.com/`
7. BeautifulSoup Web Scraping: `https://www.crummy.com/software/BeautifulSoup/`
8. Reddit API: `https://www.reddit.com/dev/api/`
9. Twitter API (Tweepy): `https://docs.tweepy.org/en/stable/`